

File Permissions in Linux

Understanding permission types :

- 1. **r** (read) -> Allows viewing the contents of a file, or listing the contents of a directory.
- 2. **w** (write) -> Allows modifying or deleting a file, or adding/removing files inside a directory.
- 3. **x** (execute) -> Allows running a file as a program, or entering (cd into) a directory.
- 4. **owner** -> The user who created the file, has their own set of permissions.
- 5. **group** -> The group assigned to the file, has its own set of permissions.
- 6. **others** -> Everyone else on the system, has its own set of permissions.

In Simple Words :

- 1. r -> read
- 2. w -> write
- 3. x -> execute
- 4. owner -> file's creator
- 5. group -> assigned group
- 6. others -> everyone else

Reading permission strings :

- 1. **-rwxr-xr--** -> First character shows file type (- for file, d for directory), followed by three sets of 3: owner, group, others.
- 2. **rwx** (owner) -> Owner can read, write, and execute.
- 3. **r-x** (group) -> Group can read and execute, but not write.
- 4. **r--** (others) -> Others can only read.

In Simple Words :

- 1. 1st letter -> file (-) or directory (d)
- 2. next 3 -> owner permissions
- 3. next 3 -> group permissions
- 4. last 3 -> others permissions

chmod command (change permissions) :

- 1. **chmod 755 file** -> Owner gets read/write/execute, group and others get read/execute (numeric method).
- 2. **chmod 644 file** -> Owner gets read/write, group and others get read only.
- 3. **chmod u+x file** -> Adds execute permission for the owner only (symbolic method).
- 4. **chmod g-w file** -> Removes write permission from the group.
- 5. **chmod o=r file** -> Sets others' permission to read only, removing anything else.
- 6. **chmod a+x file** -> Adds execute permission for owner, group, and others all at once.
- 7. **chmod -R 755 folder** -> Applies permissions recursively to a folder and everything inside it.

In Simple Words :

- 1. chmod -> change permissions
- 2. numeric (755, 644) -> quick way using numbers
- 3. u -> owner (user)
- 4. g -> group
- 5. o -> others
- 6. a -> all (owner+group+others)
- 7. + -> add permission
- 8. - -> remove permission
- 9. = -> set exact permission
- 10. -R -> apply recursively

Numeric permission values :

- 1. **4** -> read (r)
- 2. **2** -> write (w)
- 3. **1** -> execute (x)
- 4. **7** -> read+write+execute (4+2+1)
- 5. **6** -> read+write (4+2)
- 6. **5** -> read+execute (4+1)
- 7. **0** -> no permissions

In Simple Words :

- 1. 4 -> read
- 2. 2 -> write
- 3. 1 -> execute
- 4. add numbers together for combined permissions

chown command (change ownership) :

- 1. **sudo chown user file** -> Changes the owner of a file to the specified user.
- 2. **sudo chown user:group file** -> Changes both the owner and the group of a file in one go.
- 3. **sudo chown :group file** -> Changes only the group, leaving the owner unchanged.
- 4. **sudo chown -R user:group folder** -> Changes ownership recursively for a folder and its contents.

In Simple Words :

- 1. chown -> change owner
- 2. user:group -> set both owner and group
- 3. :group -> set group only
- 4. -R -> apply recursively

chgrp command (change group only) :

- 1. **sudo chgrp group file** -> Changes only the group of a file without touching the owner.
- 2. **sudo chgrp -R group folder** -> Changes the group recursively for a folder and everything inside it.

In Simple Words :

- 1. chgrp -> change group
- 2. -R -> apply recursively

Special permissions :

- 1. **chmod u+s file** -> Sets the SUID bit, so the file runs with the owner's permissions instead of the person running it.
- 2. **chmod g+s folder** -> Sets the SGID bit, so new files created inside the folder inherit its group.
- 3. **chmod +t folder** -> Sets the sticky bit, so only the owner of a file can delete it even if others have write access to the folder (commonly used on /tmp).

In Simple Words :

- 1. SUID -> run as file owner
- 2. SGID -> new files inherit folder's group
- 3. sticky bit -> only owner can delete their own files